



Bericht in

Entwicklung einer Metaheuristik für das Job Shop
Scheduling Problem unter Berücksichtigung von
reihenfolgeabhängigen Rüstzeiten

Jakob Weber Matr.-Nr. 3869088

Johannes Walz Matr.-Nr. 3533824

Prüfer: Prof.Dr.David Müller

Inhaltsverzeichnis

1	Einleitung	2
1.1	Problemstellung	2
1.2	Zielsetzung	2
1.3	Aufbau der Arbeit	3
2	Notation	4
3	Methodik	7
3.1	Eröffnungsheuristik - GT-Algorithmus	7
3.2	Metaheuristik - Tabu-Suche	9
3.2.1	Kritischer Pfad und kritische Blöcke	10
3.2.2	Nachbarschaftsdefinitionen	12
3.3	Hard- und Softwarespezifikationen	13
4	Ergebnisse	14
5	Diskussion	16
A	Anhang	21

Kapitel 1

Einleitung

1.1 Problemstellung

In dieser Arbeit wird eine Metaheuristik für das Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten entwickelt und evaluiert. Dieses Problem gehört zur Klasse der NP-schweren Optimierungsprobleme. Die Problemstellung befasst sich mit der zeitlichen Einplanung einer Menge von Jobs auf einer festgelegten Menge von Maschinen. Jeder Job besteht aus einer strikten Abfolge von einzelnen Operationen. Die technologische Reihenfolge dieser Operationen ist durch die Problemstellung fest vorgegeben. Eine Maschine kann zu jedem Zeitpunkt höchstens eine Operation gleichzeitig bearbeiten.

Eine wesentliche praxisrelevante Erweiterung stellen die sequenzabhängigen Rüstzeiten dar. Diese Rüstzeiten fallen an, wenn zwei Operationen unterschiedlicher Jobs direkt nacheinander auf derselben Maschine verarbeitet werden. Die Dauer des Rüstvorgangs hängt dabei explizit von der Reihenfolge der beiden Jobs ab. Das primäre Ziel der Problemstellung ist die Ermittlung eines zulässigen Ablaufplans. Hierbei müssen die optimalen Startzeiten für alle Operationen bestimmt werden. Die Zielfunktion verlangt die Minimierung der Gesamtproduktionsdauer, auch als Makespan $C_{\max}$ bezeichnet. Durch die zusätzlichen Rüstzeiten vergrößert sich der kombinatorische Suchraum erheblich. Exakte Lösungsverfahren stoßen daher bei größeren Probleminstanzen schnell an ihre Grenzen.

1.2 Zielsetzung

Die primäre Zielsetzung dieser Arbeit ist die Entwicklung und Evaluierung einer Metaheuristik zur Lösung des Job-Shop-Scheduling-Problems mit sequenzabhängigen Rüstzeiten. Als algorithmischer Ansatz wird hierfür eine Tabu-Suche implementiert. Im Zentrum der Untersuchung steht die Beantwortung der Frage, wie sich die Wahl der Nachbarschaftsdefinition auf

den resultierenden Makespan auswirkt. Dazu werden drei unterschiedliche Nachbarschaftsstrukturen systematisch analysiert und miteinander verglichen.

Konkret betrifft dies die Adjacent-Swap-, die N5- und die N6-Nachbarschaft. Ein wichtiges Teilziel ist zudem die Implementierung einer geeigneten Eröffnungsheuristik. Hierfür wird der Giffler-Thompson-Algorithmus genutzt, um zulässige Startlösungen für das Suchverfahren bereitzustellen. Zur Validierung der Leistungsfähigkeit wird die Tabu-Suche auf ein vorgegebenes Testset angewendet.

Die Ergebnisse werden anschließend einer quantitativen Analyse unterzogen. Als Performance-Referenz dient dabei ein exakter CP-Solver auf Basis der Google OR-Tools. Ziel ist es, die Stärken und Schwächen der jeweiligen Nachbarschaften hinsichtlich der Lösungsqualität und der benötigten Rechenzeit systematisch offenzulegen.

1.3 Aufbau der Arbeit

Diese Arbeit gliedert sich in fünf Kapitel. Nach der Einleitung führt Kapitel 2 die mathematische Notation für das betrachtete Job-Shop-Scheduling-Problem ein. Diese dient als formale Grundlage für die Algorithmen und die zugehörige Bewertungslogik. Dabei werden die Symbole für das Kernproblem, die Eröffnungsheuristik und die Tabu-Suche definiert.

Kapitel 3 beschreibt die methodische Umsetzung der Lösungsverfahren. Zunächst wird der Giffler-Thompson-Algorithmus zur Erzeugung zulässiger Startlösungen vorgestellt.

Anschließend wird die Funktionsweise der Tabu-Suche dargelegt. Dies umfasst die Berechnung des kritischen Pfades, die Extraktion kritischer Blöcke sowie die Definition der Nachbarschaften Adjacent-Swap, N5 und N6. Den Abschluss bilden die Hard- und Softwarespezifikationen.

Kapitel 4 präsentiert die Ergebnisse der Evaluation. Hierbei wird zunächst die Leistung der verschiedenen Prioritätsregeln ausgewertet. Zudem werden die drei Nachbarschaftsdefinitionen quantitativ miteinander verglichen. Die Leistungsfähigkeit der Metaheuristik wird schließlich durch den Vergleich mit einem exakten CP-Solver validiert.

Kapitel 5 schließt die Arbeit mit einer kritischen Diskussion der Ergebnisse ab. Dabei werden die Stärken und Schwächen der jeweiligen Nachbarschaften hinsichtlich Lösungsqualität und Rechenzeit systematisch offengelegt.

Kapitel 2

Notation

Notation der Job-Shop-Scheduling-Problems mit sequenzabhängigen Rüstzeiten

In diesem Kapitel wird die Notation für das betrachtete Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten eingeführt. Die Notation dient als Grundlage für die Beschreibung der Tabu-Search-Heuristik und der verwendeten Bewertungslogik.

Gegeben ist eine Menge von Jobs J und eine Menge von Maschinen M . Jeder Job $j \in J$ besteht aus einer geordneten Folge von Operationen

$$O_j = (o_{j,1}, o_{j,2}, \dots, o_{j,q_j}).$$

Die Reihenfolge der Operationen innerhalb eines Jobs ist vorgegeben. Eine Operation $o_{j,k}$ darf daher erst beginnen, wenn ihre direkte Vorgängeroperation $o_{j,k-1}$ abgeschlossen ist.

Jede Operation $o_{j,k}$ wird auf genau einer Maschine $m_{j,k} \in M$ bearbeitet und besitzt eine Bearbeitungszeit $p_{j,k}$. Zwischen zwei Operationen, die auf derselben Maschine direkt aufeinanderfolgen, kann eine sequenzabhängige Rüstzeit $s_{u,v}$ anfallen. Diese hängt von der vorherigen Operation u und der nachfolgenden Operation v ab.

Tabelle 1: Notation des Job-Shop-Scheduling-Problems

Symbol	Bedeutung
J	Menge der Jobs
j	Index eines Jobs, $j \in J$
M	Menge der Maschinen
m	Index einer Maschine, $m \in M$
O_j	geordnete Menge der Operationen von Job j
$o_{j,k}$	k -te Operation von Job j
q_j	Anzahl der Operationen von Job j
O	Menge aller Operationen
$m_{j,k}$	Maschine, auf der Operation $o_{j,k}$ bearbeitet wird
$p_{j,k}$	Bearbeitungszeit der Operation $o_{j,k}$
$s_{u,v}$	Rüstzeit zwischen zwei direkt aufeinanderfolgenden Operationen u und v
$t(o)$	Startzeit der Operation o
$C(o)$	Endzeit der Operation o
$C_{\max}$	Makespan einer Lösung
$F(S)$	Zielfunktionswert der Lösung S

Notation der Eröffnungsheuristik

Zur Erzeugung einer zulässigen Startlösung wird der Giffler-Thompson-Algorithmus verwendet. Für die mathematische Formulierung dieses Verfahrens werden folgende zusätzliche Symbole und Hilfsvariablen benötigt:

Tabelle 2: Notation der Eröffnungsheuristik (Giffler-Thompson-Algorithmus)

Symbol	Bedeutung
U	Menge der noch unverplanten Operationen, $U \subseteq O$
L_m	aktuelle Last der Maschine $m \in M$
$\lambda(m)$	Index des zuletzt auf Maschine m eingeplanten Jobs
$r(o)$	frühestmöglicher Startzeitpunkt von Operation o
$r_m(o)$	frühestmöglicher Maschinen-Startzeitpunkt von Operation o
C^*	minimaler frühestmöglicher Endzeitpunkt der aktuellen Iteration
m^*	kritische Maschine, auf der C^* erreicht wird
K	Konfliktmenge einplanbarer Operationen auf Maschine m^*
π	angewendete Prioritätsregel

Notation der Tabu-Suche

Zur Lösung des Problems wird eine Tabu-Search-Heuristik verwendet. Die aktuelle Lösung wird mit S bezeichnet. Sie wird im Vorfeld durch den Giffler-Thompson-Algorithmus erzeugt. Die beste bisher gefundene Lösung wird als S^* notiert. Die Nachbarschaft der aktuellen Lösung wird durch $NB(S)$ beschrieben. Sie enthält alle Lösungen, die durch zulässige lokale Bewegungen aus S erzeugt werden können.

Die Tabu-Liste TL speichert Bewegungsattribute, die für eine begrenzte Anzahl von Iterationen tabu sind. Dadurch wird ein unmittelbares Zurückkehren in vorherige Lösungen verhindert. Tabuierte Bewegungen können durch das Aspirationskriterium zugelassen werden, wenn sie zu einer neuen globalen Bestlösung führen.

Tabelle 3: Notation der Tabu-Search-Heuristik

Symbol	Bedeutung
$\hat{S}$	erzeugte Startlösung
S	aktuelle Lösung
S^*	beste bisher gefundene Lösung
S'	Nachbarlösung der aktuellen Lösung
$NB(S)$	Nachbarschaft der aktuellen Lösung S
TL	Tabu-Liste
TL^{Dauer}	Gültigkeitsdauer eines Tabu-Eintrags
I	Anzahl der Iterationen ohne Verbesserung
$I^{\max}$	maximale Anzahl an Iterationen ohne Verbesserung
I_{total}	Gesamtzahl der Iterationen
$I_{\text{total}}^{\max}$	maximale Gesamtzahl an Iterationen
$I^{\text{Diversifikation}}$	Schwelle für zusätzliche Diversifikationsbewegungen
$\tau^{\max}$	optionales Zeitlimit
$F(S)$	Zielfunktionswert der Lösung S

Da der Makespan minimiert wird, gilt äquivalent:

$$C_{\max}(S') < C_{\max}(S^*).$$

Kapitel 3

Methodik

3.1 Eröffnungsheuristik - GT-Algorithmus

Algorithm 1 Giffler-Thompson-Algorithmus

```
1:  $U \leftarrow O$ 
2:  $L_m \leftarrow 0 \quad \forall m \in M$ 
3:  $\lambda(m) \leftarrow \emptyset \quad \forall m \in M$ 
4:  $r(o_{j,1}) \leftarrow 0 \quad \forall j \in J$ 
5:  $r(o_{j,k}) \leftarrow \infty \quad \forall j \in J, k > 1$ 
6: while  $U \neq \emptyset$  do
7:   Berechne für alle  $o_{j,k} \in U$  mit  $r(o_{j,k}) < \infty$  den frühesten Startzeitpunkt:
      $r_m(o_{j,k}) \leftarrow \max\{L_{m_{j,k}} + s_{\lambda(m_{j,k}),j}, r(o_{j,k})\}$ 
8:    $C^* \leftarrow \min\{r_m(o_{j,k}) + p_{j,k} \mid o_{j,k} \in U \text{ und } r(o_{j,k}) < \infty\}$ 
9:   Sei  $m^*$  die Maschine, auf der  $C^*$  erreicht wird
10:  Bilde Konfliktmenge  $K \leftarrow \{o_{j,k} \in U \mid m_{j,k} = m^*, r(o_{j,k}) < \infty, r_m(o_{j,k}) < C^*\}$ 
11:  Wähle Operation  $o' = o_{j',k'} \in K$  gemäß der Prioritätsregel  $\pi$ 
12:   $t(o') \leftarrow r_m(o')$ 
13:   $C(o') \leftarrow t(o') + p_{j',k'}$ 
14:   $L_{m^*} \leftarrow C(o')$ 
15:   $\lambda(m^*) \leftarrow j'$ 
16:  if  $k' < q_{j'}$  then
17:     $r(o_{j',k'+1}) \leftarrow C(o')$ 
18:  end if
19:   $U \leftarrow U \setminus \{o'\}$ 
20: end while
```

Der Giffler-Thompson-Algorithmus [1] wird als Eröffnungsheuristik zur Generierung einer zulässigen Startlösung für das Job Shop Scheduling Problem verwendet.

Der Algorithmus beginnt mit der Initialisierung der Menge U aller unverplanten Operationen, die zu Beginn der Gesamtmenge O entspricht. Die Maschinenlasten L_m sowie die Historie der zuletzt auf einer Maschine eingeplanten Jobs $\lambda(m)$ werden für alle Maschinen $m \in M$ auf Null bzw. die leere Menge $\emptyset$ gesetzt. Die Bereitschaftszeit $r(o_{j,k})$ wird für die jeweils ersten Operationen aller Jobs auf Null und für alle nachfolgenden Operationen auf unendlich initialisiert, um zu signalisieren, dass diese noch nicht einplanbar sind.

In jeder Iteration der Hauptschleife wird für alle einplanbaren Operationen in U der frühestmögliche Maschinen-Startzeitpunkt $r_m(o_{j,k})$ berechnet. Dieser ergibt sich als Maximum aus der aktuellen Last der benötigten Maschine zuzüglich einer eventuell anfallenden Rüstzeit $s_{\lambda(m_{j,k}),j}$ und der Bereitschaftszeit $r(o_{j,k})$ der Operation selbst. Darauf aufbauend wird der minimale Endzeitpunkt C^* über alle einplanbaren Operationen bestimmt. Die Maschine, auf der dieser Wert erreicht wird, wird als kritische Maschine m^* definiert.

Anschließend wird die Konfliktmenge K gebildet. Diese enthält alle noch nicht verplanten Operationen auf der kritischen Maschine m^* , deren frühestmöglicher Startzeitpunkt echt kleiner als C^* ist. Aus dieser Menge wird mithilfe einer Prioritätsregel π eine Operation $o' = o_{j',k'}$ zur festen Einplanung ausgewählt.

Das Einplanen erfolgt durch das Setzen der Startzeit $t(o')$ auf ihren frühestmöglichen Maschinen-Startzeitpunkt $r_m(o')$ und der Berechnung des Fertigstellungszeitpunkts $C(o')$. Daraufhin werden die Last L_{m^*} der kritischen Maschine auf $C(o')$ aktualisiert und der Job-Index j' als der zuletzt auf m^* verarbeitete Job gespeichert. Falls eine Nachfolgeoperation $o_{j',k'+1}$ im selben Job existiert, wird deren Bereitschaftszeit $r(o_{j',k'+1})$ auf die Endzeit $C(o')$ angehoben, womit sie in der nächsten Iteration einplanbar wird. Abschließend wird die geplante Operation o' aus der Menge U entfernt. Der Algorithmus terminiert, sobald alle Operationen eingeplant wurden und somit $U = \emptyset$ gilt.

Da in jeder Iteration des Algorithmus Konflikte zwischen einplanbaren Operationen auf derselben Maschine auftreten können, werden Prioritätsregeln eingesetzt, um diese Konflikte aufzulösen. Die Wahl der Prioritätsregel hat dabei einen maßgeblichen Einfluss auf die Qualität des resultierenden Ablaufplans (den Makespan $C_{\max}$).

Zu Beginn wurden acht verschiedene Prioritätsregeln für das Problem untersucht. Zunächst einfache Regeln, die sich auf eine einzelne Eigenschaft einer Operation beziehen.

1. **Shortest Processing Time (SPT)**, dass die Operationen nach aufsteigender Prozessdauer sortiert
2. **Shortest Processing Time + Setup (SPTS)**, dass zusätzlich zur Prozessdauer die Rüstzeit betrachtet

3. **Longest Processing Time (LPT)**, dass Operationen nach absteigender Prozessdauer sortiert
4. **First Job (FJ)**, dass die Operationen nach ihrem Index sortiert

Hier weitere Prio Regeln einfügen und erklären

3.2 Metaheuristik - Tabu-Suche

Algorithm 2 Tabu Search für das Job-Shop-Scheduling-Problem mit sequenzabhängigen Rüstzeiten

```

1:  $S \leftarrow \hat{S}$ 
2:  $S^* \leftarrow S$ 
3:  $TL \leftarrow \emptyset$ 
4:  $I_{\text{total}} \leftarrow 0$ 
5:  $I \leftarrow 0$ 
6:  $\tau \leftarrow 0$ 
7: while  $I_{\text{total}} < I_{\text{total}}^{\max}$  and  $I < I^{\max}$  and  $\tau < \tau^{\max}$  do
8:    $I_{\text{total}} \leftarrow I_{\text{total}} + 1$ 
9:   Erzeuge die Nachbarschaft  $NB(S)$  der aktuellen Lösung
10:  if  $I \geq I^{\text{div}}$  then
11:    Ergänze zufällige nicht-kritische Bewegungen zu  $NB(S)$ 
12:  end if
13:  for  $S' \in NB(S)$  do (jede Nachbarlösung)
14:    if  $S'$  ist unzulässig or (tabuiert and Aspiration nicht erfüllt) then
15:      Entferne  $S'$  aus  $NB(S)$ 
16:    end if
17:  end for
18:  Wähle die beste zulässige Nachbarlösung  $S' \in NB(S)$ 
19:   $S \leftarrow S'$ 
20:  Entferne abgelaufene Einträge aus  $TL$ 
21:   $TL \leftarrow TL \cup \{\text{inverse Bewegung von } S'\}$ 
22:  if  $F(S) < F(S^*)$  then
23:     $S^* \leftarrow S$ 
24:     $I \leftarrow 0$ 
25:  else
26:     $I \leftarrow I + 1$ 
27:  end if
28: end while
29: return  $S^*$ 

```

Die Suche startet mit einer zulässigen Startlösung S . Die bisher beste Lösung wird mit $S^* \leftarrow S$ initialisiert. Zusätzlich werden Tabu-Liste TL , Gesamtiterationszähler I_{total} und Verbesserungszähler I initialisiert. Die Abbruchbedingungen sind maximale Gesamtiterationen, maximale Iterationen ohne Verbesserung und ein optionales Zeitlimit.

Die Nachbarschaft $NB(S)$ wird in jeder Iteration aus einer als Parameter übergebenen Nachbarschaftsdefinition erzeugt. Zur Auswahl stehen Adjacent Swap in kritischen Blöcken, N5 und N6. Die gewählte Definition bestimmt Größe und Struktur der Kandidatenmenge.

Eine Nachbarlösung wird vor der Bewertung auf Zulässigkeit geprüft. Unzulässig sind Kandidaten, deren Bewegung im Dispositionsgraphen einen Zyklus erzeugt. Diese Kandidaten werden sofort verworfen.

Die Tabu-Tenure ist fest auf $T_{\text{tabu}} = 20$ Iterationen gesetzt. Beim Eintrag einer inversen Bewegung in TL wird die Verbotsdauer als Ablaufiteration gespeichert. Abgelaufene Einträge werden aus Effizienzgründen nicht in jeder Iteration entfernt, sondern periodisch alle 32 Iterationen bereinigt.

Vor der Auswahl wird die gesamte Kandidatenliste $NB(S)$ zufällig permutiert. Anschließend wird die erste beste gefundene zulässige Nachbarlösung übernommen. Dadurch entsteht bei mehreren gleichwertigen Kandidaten eine zufällige Tie-Break-Regel.

Tabuierte Bewegungen werden nur akzeptiert, wenn das Aspirationskriterium erfüllt ist. Die Bedingung lautet $F(S') < F(S^*)$. Damit bezieht sich die Aspiration auf die bisher beste Lösung und nicht auf die aktuelle Lösung.

Bleibt nach Zulässigkeits- und Tabu-Filterung kein Kandidat übrig, wird einmalig eine Menge zufälliger nicht-kritischer Bewegungen erzeugt. Dieser Fallback erfolgt unabhängig von I^{div} . Die zusätzlichen Kandidaten werden erneut auf Zulässigkeit und Tabu-Status geprüft. Erst wenn auch dann keine zulässige Bewegung existiert, wird die Suche beendet.

Wird eine Nachbarlösung S' akzeptiert, wird $S \leftarrow S'$ gesetzt und die inverse Bewegung in TL eingetragen. Falls $F(S) < F(S^*)$ gilt, wird $S^* \leftarrow S$ gesetzt und $I \leftarrow 0$. Andernfalls wird $I \leftarrow I + 1$ gesetzt. Nach Abschluss der Suche wird S^* zurückgegeben.

3.2.1 Kritischer Pfad und kritische Blöcke

Die Bewertung einer Lösung erfolgt auf Basis eines disjunktiven Graphen. Eine solche Modellierung kann auf die Arbeiten von Balas (1969) [2] zurückgeführt werden. Jede Operation wird durch einen Knoten repräsentiert. Zwischen den Knoten existieren konjunktive Kanten für die technologische Reihenfolge innerhalb eines Jobs sowie disjunktive Kanten für die Bearbeitungsreihenfolge auf einer Maschine. Die konjunktiven Kanten sind durch die Jobs der Instanz vorgegeben. Wohingegen die Richtung der disjunktiven Kanten zunächst offen und erst durch die in der aktuellen Lösung festgelegten Maschinenreihenfolgen bestimmt wird.

Diese Kanten tragen die Bearbeitungsdauer und die zusätzlichen sequenzabhängige Rüstzeiten. Sobald für jede Maschine eine zulässige Reihenfolge vorliegt, ist der resultierende Graph azyklisch.

In dem azyklischen Graphen entspricht die früheste Startzeit einer Operation der Länge des längsten Pfades von einer Quelle zu diesem Knoten. Die Funktion `evaluate_orders(problem, orders)` berechnet diese Startzeiten durch eine Longest-Path-Berechnung. Zunächst wird für jede Operation der Eingangsgrad bestimmt. Eine Operation besitzt höchstens zwei unmittelbare Vorgänger: einen Job-Vorgänger und einen Maschinen-Vorgänger. Anschließend werden alle Operationen ohne Vorgänger mit Startzeit null in eine Warteschlange eingefügt. Bei der Verarbeitung einer Operation u werden ihre Nachfolger entlang beider Kantentypen relaxiert. Für jeden Nachfolger v gilt

$$start(v) = \max(start(v), completion(u) + s_{uv}),$$

wobei s_{uv} die zusätzliche Rüstzeit auf der betrachteten Maschinenkante bezeichnet und auf konjunktiven Kanten null ist. Die Maximumbildung stellt sicher, dass eine Operation erst startet, wenn alle unmittelbaren Vorgänger abgeschlossen sind. Erhöht eine Relaxation den Startzeitwert eines Nachfolgers, wird der auslösende Vorgänger als bindender Vorgänger gespeichert.

Werden im Verlauf der topologischen Verarbeitung nicht alle Operationen besucht, enthält der disjunktive Graph einen Zyklus. Eine solche Reihenfolge ist unzulässig und wird verworfen. Nach Abschluss der Berechnung ergibt sich der Makespan $C_{\max}$ als maximale Fertigstellungszeit über alle Operationen. Der kritische Pfad wird anschließend durch Rückverfolgung der gespeicherten bindenden Vorgänger konstruiert. Ausgehend von einer Operation mit Fertigstellungszeit $C_{\max}$ wird iterativ zum jeweiligen Vorgänger zurückgegangen, bis eine Quelle erreicht ist. Die umgekehrte Folge dieser Operationen bildet einen kritischen Pfad. Die Summe seiner Kantengewichte entspricht exakt dem Makespan.

Auf dieser Grundlage werden kritische Blöcke bestimmt. Ein kritischer Block ist eine maximale Teilfolge des kritischen Pfades, deren Operationen auf derselben Maschine liegen und dort unmittelbar aufeinanderfolgen. Diese Blöcke definieren den Suchraum der kritischkeitsbasierten Nachbarschaften [3].

Die aktuelle Implementierung liefert bei mehreren gleich langen kritischen Pfaden nur einen einzelnen Pfad zurück. Dies folgt erstens aus der Wahl des Endknotens, da bei mehreren Operationen mit identischer maximaler Fertigstellungszeit nur der erste gefundene Endpunkt verwendet wird. Zweitens wird bei Gleichstand konkurrierender Vorgänger nur die zuerst verarbeitete bindende Kante gespeichert. Der berechnete Makespan bleibt dadurch korrekt. Eingeschränkt ist jedoch die nachgelagerte Nachbarschaftserzeugung, weil kritische Blöcke ausschließlich aus diesem einen Pfad extrahiert werden. Existieren mehrere voneinander un-

abhängige kritische Pfade, können erzeugte Bewegungen einen sichtbaren Pfad verändern, ohne den Makespan zu reduzieren, weil ein weiterer unbeachteter Pfad weiterhin bindend bleibt. Im vorliegenden Verfahren wird dieser Effekt nur teilweise abgeschwächt. Zusätzliche nicht-kritische Diversifikationszüge können nach längerer Stagnation indirekt einen weiteren kritischen Pfad treffen. Außerdem kann sich durch eine geänderte Reihenfolge in einer späteren Iteration ein anderer gleich langer Pfad als kritisch manifestieren. Eine vollständige Behandlung würde die gleichzeitige Identifikation aller Endoperationen mit Fertigstellungszeit $C_{\max}$ und die Berücksichtigung mehrerer kritischer Pfade in der Blockbildung erfordern.

3.2.2 Nachbarschaftsdefinitionen

Für die Tabu-Suche werden drei Nachbarschaftsdefinitionen verwendet, die auf dem kritischen Pfad basieren. Der kritische Pfad ist die Folge von Operationen, die den Makespan $C_{\max}$ bestimmt. Eine Verzögerung einer Operation auf diesem Pfad erhöht unmittelbar die Gesamtdauer. [4]

Zur Erzeugung von Bewegungen wird der kritische Pfad in kritische Blöcke zerlegt. Ein kritischer Block ist eine maximale Teilfolge von Operationen, die auf dem kritischen Pfad liegen und auf derselben Maschine unmittelbar aufeinanderfolgen. Die Funktion `critical_blocks()` durchläuft den kritischen Pfad und fasst Operationen so lange zu einem Block zusammen, wie Maschinenzuordnung und unmittelbare Nachbarschaft erhalten bleiben. Bei Maschinenwechsel oder fehlender Adjazenz wird ein neuer Block begonnen. Blöcke mit weniger als zwei Operationen werden verworfen, da kein sinnvoller Tausch definiert ist. [3]

Die Einschränkung auf Operationen innerhalb kritischer Blöcke folgt der klassischen Eigenschaft, dass Verbesserungen des Makespans nur durch Änderungen innerhalb solcher Blöcke erreicht werden. Vertauschungen über Blockgrenzen hinweg reduzieren den Makespan nicht. [5]

Die Implementierung umfasst drei Nachbarschaften.

Adjacent-Swap-Nachbarschaft. Für jeden kritischen Block werden alle benachbarten Operationspaare vertauscht. Ein Block $[A, B, C, D]$ erzeugt die Züge (A, B) , (B, C) und (C, D) . Diese Variante betrachtet alle lokalen Adjazenzvertauschungen innerhalb kritischer Blöcke. [5]

N5-Nachbarschaft. Pro kritischem Block werden nur die ersten beiden sowie die letzten beiden Operationen vertauscht. Ein Block $[A, B, C, D, E]$ erzeugt damit ausschließlich die Züge (A, B) und (D, E) . Die Reduktion basiert auf dem Ergebnis, dass Randvertauschungen makespanwirksam sein können, während innere Vertauschungen diese Eigenschaft nicht besitzen. [3]

N6-Nachbarschaft. N6 erweitert N5 um Insert-Bewegungen. Zusätzlich zu den N5-Randvertauschungen kann jede innere Operation eines Blocks an den Blockanfang oder an das Blockende verscho-

ben werden. Für den Block $[A, B, C, D, E]$ entstehen neben (A, B) und (D, E) beispielsweise die Sequenzen $[C, A, B, D, E]$ und $[A, C, D, E, B]$. Duplikate werden ausgeschlossen, wenn ein Insert-Zug äquivalent zu einem bereits enthaltenen N5-Zug ist. Diese Erweiterung vergrößert die strukturelle Diversität der Nachbarschaft bei weiterhin begrenzter Kandidatenzahl. [6]

3.3 Hard- und Softwarespezifikationen

Die Instanzen wurden auf einem Apple MacBook Pro aus dem Jahr 2023 ausgeführt. Das System ist mit einem Apple M3 Pro Prozessor sowie 18 GB gemeinsam genutztem Arbeitsspeicher ausgestattet. Die Berechnungen wurden lokal auf dem Gerät durchgeführt, ohne zusätzliche externe Rechenressourcen oder Serverinfrastruktur zu verwenden.

Die algorithmische Umsetzung erfolgte in der Programmiersprache Python in der Version 3.9.7. Als Betriebssystem kam macOS Tahoe 26.5.2 zum Einsatz. Für die exakte Validierung wurde der CP-Solver aus den Google OR-Tools in der Version 9.15.6755 verwendet. Für mathematische Operationen und Datenstrukturen wurde zudem die Bibliothek NumPy eingebunden. Die Erstellung der Gantt-Charts zur Visualisierung der Ablaufpläne erfolgte mithilfe von Google Charts.

Kapitel 4

Ergebnisse

Tabelle 1: Ergebnisse der Giffler-Thompson-Prioritätsregeln

- Prioritätsregel- Cmax- Laufzeit- Anzahl Rüstvorgänge oder gesamte Rüstzeit, falls relevant

In Abbildung 1 werden die verschiedenen implementierten Prioritätsregeln (wie SPT, LPT, MINSTART, MINEND sowie kombinierte Heuristiken) über ausgewählte Instanzen hinweg verglichen.

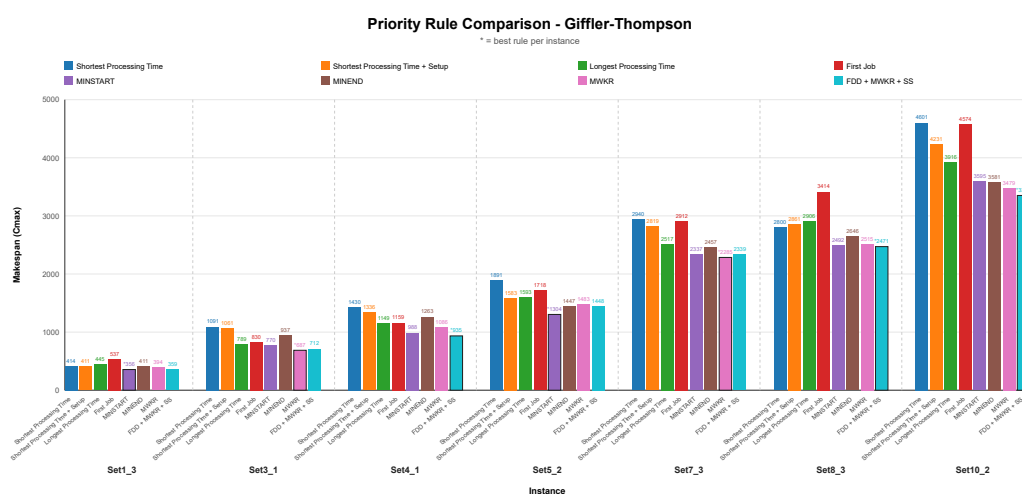


Abbildung 1: Vergleich der prioritätsbasierten Eröffnungsheuristiken anhand des erreichten Makespan $C_{\max}$ für ausgewählte Classroom-Instanzen.

Tabelle 2: Vergleich der Tabu-Search-Nachbarschaften

- Nachbarschaft- bester Cmax - durchschnittlicher Cmax- schlechter Cmax - Standardabweichung- durchschnittliche Laufzeit- Anzahl Iterationen

Tabelle 3: Vergleich mit OR-Tools

- Instanz - OR-Tools Cmax - Tabu Search Cmax - absolute Abweichung - relative Abweichung
in % - Laufzeit OR-Tools - Laufzeit Tabu Search

Kapitel 5

Diskussion

Abbildungsverzeichnis

1	Vergleich der prioritätsbasierten Eröffnungsheuristiken anhand des erreichten Makespans $C_{\max}$ für ausgewählte Classroom-Instanzen.	14
---	---	----

Quellcodeverzeichnis

Tabellenverzeichnis

1	Notation des Job-Shop-Scheduling-Problems	5
2	Notation der Eröffnungsheuristik (Giffler-Thompson-Algorithmus)	5
3	Notation der Tabu-Search-Heuristik	6

Literaturverzeichnis

- [1] B. Giffler und G. L. Thompson, „Algorithms for solving production-scheduling problems,” *Operations Research*, Vol. 8, Nr. 4, S. 487–503, 1960. (Zitiert auf Seite 8.)
- [2] E. Balas, „Machine Sequencing via Disjunctive Graphs: An Implicit Enumeration Algorithm,” *Operations Research*, Vol. 17, Nr. 6, S. 941–957, 1969. [Online]. Available: <http://www.jstor.org/stable/168317> (Zitiert auf Seite 10.)
- [3] E. Nowicki und C. Smutnicki, „A Fast Taboo Search Algorithm for the Job Shop Problem,” Vol. 42, Nr. 6, S. 797–813. [Online]. Available: <http://www.jstor.org/stable/2634595> (Zitiert auf Seiten 11 und 12.)
- [4] J. Blazewicz, K. H. Ecker, E. Pesch, G. Schmidt, M. Sterna, und J. Weglarz, *Handbook on Scheduling: From Theory to Practice*. Springer International Publishing. [Online]. Available: <http://link.springer.com/10.1007/978-3-319-99849-7> (Zitiert auf Seite 12.)
- [5] Peter J. M. van Laarhoven, Emile H. L. Aarts, und J. K. Lenstra, „Job Shop Scheduling by Simulated Annealing,” Vol. 40, Nr. 1, S. 113–125. [Online]. Available: <http://www.jstor.org/stable/171189> (Zitiert auf Seite 12.)
- [6] E. Balas und A. Vazacopoulos, „Guided Local Search with Shifting Bottleneck for Job Shop Scheduling,” *Management Science*, Vol. 44, Nr. 2, S. 262–275, 1998. [Online]. Available: <http://www.jstor.org/stable/2634500> (Zitiert auf Seite 13.)

Anhang A

Anhang